Agora Protocol (Colosseum X Swarm) — Synthesized Architecture Report

Proof-of-Deliberation — Multi-Agent Orchestration on Solana

Colosseum Frontier × Canteen SWARM Hackathon

Team: Dave (ML/LLM Lead) · Josh (Infra/Deployment Lead) · Joshua Ddf (UI) **Hackathon:** Colosseum Frontier + Canteen SWARM — April 6 – May 11, 2026 **Synthesis Date:** April 8, 2026

Executive Summary

This report synthesizes findings from two independent research efforts (Report A and Report B) investigating innovative approaches to **Agora** — an on-chain multi-agent orchestration system being built for the SWARM × Colosseum Frontier hackathon. Both reports agree that the baseline plan — a static classifier routing to standard round-robin LangGraph debate — is fundamentally limited by the **martingale property** of unguided multi-agent debate. Where they diverge is in the solution architecture: Report A prioritizes pragmatic, paper-backed innovations implementable within a hackathon timeline (Thompson Sampling, FREE-MAD anti-conformity, ISP voting, Merkle verification); Report B proposes a more ambitious enterprise-grade stack (cascaded contrastive-learning routers, GAT-based adjudication, TurboQuant KV compression).

The final recommended architecture cherry-picks the best of both, grounded in a single constraint: *what can a 3-person team actually ship in 5 weeks while maximizing the Innovation (40%) and Agentic Sophistication (30%) judging criteria?*

The system is called **"Proof-of-Deliberation"** — the core primitive powering **Agora Protocol**, where every step of AI collective reasoning is verifiable, from mechanism selection rationale to individual agent arguments to final consensus.

Part 1: The Shared Foundation — Why the Baseline Fails

Both reports converge on the same core critique, which is worth internalizing before any design work begins.

1.1 The Martingale Problem

The Li et al. (NeurIPS 2025 Spotlight) paper that the baseline operationalizes actually contains a finding that undermines naive debate: multi-agent debate induces a **martingale over agent belief trajectories**. In plain terms: if agents start with a prior distribution favoring an incorrect answer, unstructured debate will circulate that probability mass rather than correct it. The empirical gains usually attributed to debate are often just the statistical ensembling effect of aggregating multiple independent outputs — functionally equivalent to majority voting.

Report B articulates this with MAD-Oracle/MAD-Conformist/MAD-Follower intervention categories (locking correct states, using majority as proxy truth, forced adoption with probability). Report A cites the ICLR 2025 empirical finding that five MAD frameworks across nine benchmarks couldn't beat Chain-of-Thought + majority voting.

Implication: The debate engine needs *structural interventions* that break the martingale — not just more rounds of prompting.

1.2 Static Classification Is the Wrong Frame

Both reports reject the logistic regression / gradient-boosted tree approach for mechanism selection. The reasoning differs slightly:

- **Report A** argues that mechanism selection is an online learning problem — the optimal mechanism depends on task characteristics that can only be partially observed upfront, and the system should learn from outcomes over time.
- **Report B** argues that mechanism selection must be *continuous* — re-evaluated at every step of execution, not just at the start.

Both are correct. The synthesized approach addresses both: an online-learning selector that can re-route mid-execution.

Part 2: Mechanism Selector — Comparative Analysis & Recommendation

2.1 Report A's Approach: Thompson Sampling + LLM Reasoning Agent

Architecture: A contextual multi-armed bandit observes task features (complexity, subjectivity, answer-space size, initial disagreement) and selects from an expanded mechanism repertoire. Thompson Sampling maintains Beta posteriors per mechanism per task category. An LLM reasoning agent wraps the bandit, providing chain-of-thought justification for each selection that gets hashed on-chain.

Strengths:

- Thompson Sampling is ~20 lines of Python. Trivially implementable on day 1.
- Naturally balances exploration/exploitation — critical for a hackathon where you have limited data.
- The LLM reasoning wrapper provides *explainable* selections (judges love explainability).
- Backed by BaRP (2025) showing 12.46%+ improvement over offline classifiers under partial feedback.
- The CoCoMaMa extension handles adding new mechanisms at runtime without retraining.

Weaknesses:

- Pre-inference only — doesn't re-evaluate mid-execution.
- Thompson Sampling posteriors need ~50-100 observations per category to converge meaningfully. In a hackathon demo, you might only run 20-30 tasks total.
- The LLM reasoning wrapper adds latency and cost per task.

2.2 Report B's Approach: Cascaded Contrastive-Learning Router (MasRouter/STRMAC)

Architecture: A multi-tiered controller that optimizes across three dimensions: (1) collaboration mode (adversarial, sequential, P2P), (2) role allocation via variational latent variables, and (3) dynamic LLM backbone assignment per agent via cost-utility function. Uses contrastive learning on historical execution paths for continuous state-aware routing via cosine similarity between evolving state embeddings and agent expertise embeddings.

Strengths:

- Continuous routing is genuinely more powerful — detecting debate stagnation and switching mechanisms mid-execution is a real advantage.
- Heterogeneous agent specialization (different LLM backbones per role) is cost-efficient and realistic.
- The cascaded dimensions (mode + roles + backbone) provide richer optimization than binary debate/vote.

Weaknesses:

- **Massive implementation cost.** Training a contrastive learning router requires: (a) a training dataset of historical execution paths, (b) expertise embeddings for every agent, (c) a reward signal. You have none of these on day 1.
- The 23.8% improvement and 52% cost reduction figures are cited without accessible references for the specific STRMAC framework.
- A variational latent variable model for role allocation is a research project unto itself.
- The "self-evolving data generation pipeline" is hand-waved — in practice, bootstrapping from zero data is the hardest part.

2.3 Synthesized Recommendation: Tiered Selector with Mid-Execution Checkpoints

The pragmatic synthesis: Start with Report A's Thompson Sampling + LLM reasoning agent, but add Report B's key insight of **continuous re-evaluation** via lightweight mid-execution checkpoints.

Implementation:

```
Phase 1 (Pre-execution): Thompson Sampling Bandit + LLM Reasoning Agent
  Input: task features (complexity score, topic category, agent count, stakes)
  Output: {mechanism: "debate"|"vote"|"delphi"|"moa", confidence: float, reasoning: str}
  → reasoning gets SHA-256 hashed and committed on-chain

Phase 2 (Mid-execution checkpoints): Lightweight State Monitor
  After each debate round or vote phase:
    - Compute disagreement entropy across agent outputs
    - Compute information gain delta from previous round
    - If entropy is rising AND information gain < threshold for 2 consecutive rounds:
        → Trigger mechanism switch (e.g., debate → vote, or escalate to Delphi)
    - Log the switch decision + justification on-chain

Phase 3 (Post-execution): Reward Update
  Bandit receives binary reward (task solved correctly / not)
  Thompson Sampling posteriors update per (task_category, mechanism) pair
```

Why this wins: You get Report A's day-1 implementability with Report B's continuous adaptation, without needing to train a contrastive learning model from scratch. The mid-execution checkpoint is just an `if` statement on two metrics you're already computing (disagreement entropy and information gain). Report B's backbone assignment idea is adopted as a simpler heuristic: use a lightweight model (e.g., Gemini 2.0 Flash or Claude Haiku) for initial generation, and a stronger model (e.g., Gemini 2.5 Pro or Claude Sonnet) for rebuttals and synthesis. In implementation, Gemini runs through Vertex AI and Claude runs through Anthropic's direct SDK, which removes Vertex quota bottlenecks for Anthropic models.

Build effort: 2-3 days total. Thompson Sampling: 1 day. LLM reasoning wrapper: 0.5 days. Mid-execution monitor: 0.5-1 day.

Part 3: Debate Engine — Comparative Analysis & Recommendation

3.1 Report A's Approach: Anti-Conformity + Tree Structure + Adaptive Termination

Key innovations:

- **FREE-MAD anti-conformity** (Cui et al., 2025): Agents identify flaws via CoT instead of conforming. Trajectory-aware scoring evaluates all intermediate outputs, not just final answers. Up to 16pp accuracy

improvement while halving token usage.

- **MACI dual-dial control** (Chang, Stanford, 2025): Information dial gates evidence quality; behavior dial schedules contentiousness. Halts when information gain plateaus for 2 consecutive rounds.
- **Tree-of-Debate** (Kargupta et al., 2025): Hierarchical argument trees instead of flat round-robin. Sub-disagreements spawn branches; resolved branches get pruned.
- **ACC-Debate** (ICLR 2025): Actor-critic training of debate behavior — debate as learned skill, not emergent prompting behavior.

3.2 Report B's Approach: TED Framework with Factional Debate + GAT Adjudication

Key innovations:

- **DebateFlow/InsightFlow separation**: Debate agents are structurally divided from adjudication agents. Debate agents are forced into opposing factions (Proponents vs. Opponents), eliminating sycophantic convergence by design.
- **Lincoln-Douglas format**: Opening statements → Cross-examination by Devil's Advocate → Iterative rebuttal with MAD-Oracle state locking → Closing statements.
- **GAT-based adjudication**: The InsightFlow Analysis Agent translates the debate into a relational graph (arguments as nodes, interactions as edges) and uses a Graph Attention Network to determine winners based on structural logical integrity, not linguistic persuasiveness.
- **TurboQuant context compression**: 3.5-bit KV cache quantization for 5x memory reduction and 8x attention speedup, enabling longer debate transcripts.

3.3 Head-to-Head Assessment

Dimension	Report A	Report B	Verdict
Anti-sycophancy	FREE-MAD's flaw-identification prompting + trajectory scoring	Factional assignment (forced opposition) + Devil's Advocate cross-examination	**Report B wins.** Structural factional separation is stronger than prompting-based anti-conformity. You can't sycophantically agree when you're assigned to oppose.
Termination logic	MACI's information-gain plateau detection	Not explicitly addressed (fixed stages)	**Report A wins.** Adaptive termination is critical for cost control and is trivial to implement.
Adjudication	Trajectory-aware scoring across all rounds	GAT on argument graph	**Report A for hackathon.** GAT requires training data, graph construction pipeline, and inference infra. Trajectory scoring is a weighted aggregation — implementable in an afternoon.
Structure	Tree-of-Debate with branch/prune	Lincoln-Douglas linear stages	**Both have merit.** Lincoln-Douglas is simpler and still very effective for 2-faction debates. Tree-of-Debate is better when there are >2 competing hypotheses.
Context management	Not directly addressed	TurboQuant KV compression	**Report B's concern is valid, but TurboQuant is overkill.** For a hackathon with 2-4 debate rounds and 3-5 agents, you won't hit context limits on Claude. Simple transcript summarization between rounds suffices.
Implementability	3-4 days	7-10 days minimum	**Report A wins on timeline.**

3.4 Synthesized Recommendation: Factional Debate with Adaptive Termination

Take Report B's structural design, strip out the over-engineering, and add Report A's termination logic.

Architecture:

```
DEBATE ENGINE v2 — "Adversarial Deliberation with Proof"

Setup Phase:
  - Agents assigned to factions: Proponents (argue FOR answer A) vs. Opponents (argue FOR
answer B)
  - Faction assignment based on initial agent outputs (top-2 answers become faction positions)
  - 1 Devil's Advocate agent (separate from factions) designated for cross-examination

Round Structure (LangGraph StateGraph with cyclic subgraph):

  Node 1: OPENING STATEMENTS [parallel fan-out via Send API]
    - Each faction agent generates independent argument for its position
    - Output: structured JSON {claim, evidence, confidence}
    - Each argument SHA-256 hashed as Merkle leaf

  Node 2: CROSS-EXAMINATION [Devil's Advocate node]
    - DA receives all opening statements
    - Generates targeted interrogation prompts for BOTH factions
    - Specifically prompted to find: logical gaps, unsupported claims, contradictions
    - Anti-conformity prompt template (FREE-MAD style):
      "Your job is to find flaws, not agree. Identify the weakest argument
       from each side and explain why it fails."

  Node 3: REBUTTAL [parallel fan-out]
    - Factions defend against cross-examination
    - MAD-Oracle integration: if any sub-claim is verified by tool call
      (e.g., code execution, math verification), that claim is STATE-LOCKED
    - Agents cannot attack locked claims; must redirect arguments

  Node 4: CONVERGENCE CHECK [conditional edge]
    - Compute: disagreement_entropy, information_gain_delta
    - IF info_gain_delta < threshold for 2 rounds → TERMINATE (MACI adaptive termination)
    - IF round_count < max_rounds AND info_gain_delta >= threshold → LOOP to Node 2
    - IF disagreement_entropy is RISING → trigger mechanism switch to voting

  Node 5: FINAL AGGREGATION
    - Trajectory-aware scoring: weight all intermediate outputs, not just final answers
    - Confidence-calibrated final answer
    - Build Merkle tree from all argument hashes → compute root
    - Commit Merkle root + decision hash to Solana

State Schema:
  {
    task: str,
    round: int,
    factions: {pro: [AgentOutput], opp: [AgentOutput]},
    cross_examinations: [DAOutput],
    rebuttals: {pro: [AgentOutput], opp: [AgentOutput]},
    locked_claims: [VerifiedClaim],
    convergence_metrics: {entropy: float, info_gain: float},
    transcript_hashes: [str],  # Merkle leaves
    final_answer: str,
    merkle_root: str
  }
```

What was dropped from Report B and why:

- GAT adjudication → replaced with trajectory-aware weighted scoring (same goal, 10x simpler)
- TurboQuant → unnecessary at hackathon scale; simple transcript summarization if needed

- DebateFlow/InsightFlow separation → kept conceptually (factions debate, separate agent adjudicates) but without the formalism of distinct sub-systems

What was kept from Report B:

- Factional assignment (structurally superior to prompting-based anti-conformity)
- Devil's Advocate cross-examination (forces genuine adversarial engagement)
- MAD-Oracle state locking (verified claims become immutable — elegant)
- Lincoln-Douglas stage structure (clear, debuggable, hashable)

What was added from Report A:

- MACI adaptive termination (plateau detection → stop wasting tokens)
- Anti-conformity prompting in the DA role (belt-and-suspenders with factions)
- Merkle tree construction from argument hashes (on-chain verification)
- Mid-execution mechanism switching (entropy-based escape hatch)

Build effort: 3-4 days.

Part 4: Vote Engine — Comparative Analysis & Recommendation

4.1 Report A's Innovations

- **Inverse Surprising Popularity (ISP)**: Agents predict both their answer AND what they think others will answer. Answers more popular than predicted get higher weight. Provably outperforms majority voting.
- **ReConcile confidence rescaling**: Raw LLM confidence is poorly calibrated. Rescale before weighting. 7.7% improvement over debate baselines.
- **Quadratic voting**: Cost = votes². Agents express preference intensity.
- **Liquid democracy**: Agents delegate to specialists.
- **Prediction-market betting**: Agents stake fictional currency. Whale bets correct ~99% of the time.

4.2 Report B's Innovations

- **Liquid democracy with fractional delegation**: Agents split voting weight across multiple specialists (e.g., 60% to Rust auditor, 40% to tokenomics expert). More granular than binary delegation.
- **Quadratic voting with reputation-weighted credit allocation**: On-chain reputation score acts as multiplier for QV credit budget.
- **Solana Agent Registry integration**: PDA-backed identity → Validation Registry (TEE oracles verify outputs) → Reputation Registry (automatic scoring/slashing after task resolution).

4.3 Assessment

Report A has more novel aggregation algorithms (ISP is genuinely innovative and underused). Report B has better on-chain integration design (reputation-weighted QV credit allocation via the Agent Registry is elegant). Neither addresses the full picture alone.

4.4 Synthesized Recommendation: Confidence-Weighted ISP with On-Chain Reputation

Core voting flow:

```
VOTE ENGINE v2 — "Calibrated Consensus with Proof"

Step 1: INDEPENDENT GENERATION [parallel fan-out]
  Each agent receives the task and produces:
```

```
  {
    answer: str,
    confidence: float,           # self-assessed confidence (0-1)
    predicted_group_answer: str,  # ISP: what they think others will say
    reasoning: str
  }

Step 2: CONFIDENCE RESCALING
  Raw LLM confidences are systematically overconfident.
  Apply temperature scaling calibration:
    calibrated_confidence = softmax(logit(raw_confidence) / T)
  where T is learned from a small calibration set (20-30 examples sufficient).

Step 3: ISP AGGREGATION
  For each unique answer A:
    actual_frequency(A) = count of agents who chose A / total agents
    predicted_frequency(A) = mean of agents' predictions for A
    surprise_score(A) = actual_frequency(A) - predicted_frequency(A)
  Answers with positive surprise_score (more popular than predicted) get boosted.
  Final weight per answer = sum(calibrated_confidence * (1 + surprise_score))
  for agents who chose that answer.

Step 4: REPUTATION WEIGHTING (on-chain)
  If Agent Registry is active:
    weight *= agent.reputation_score  # from Solana PDA
  This addresses Report B's concern about generalist hallucination outvoting specialists.

Step 5: QUORUM CHECK
  If max_weight_answer has >threshold share of total weight → quorum reached
  If not → escalate to debate mechanism (mid-execution switch)

Step 6: ON-CHAIN COMMIT
  Hash individual votes → build vote Merkle tree → commit root to Solana
  Record: {mechanism: "vote", variant: "ISP", quorum: bool, final_answer, merkle_root}
```

Why ISP over pure quadratic voting for the hackathon:

- QV requires a credit budget system, strategic allocation logic, and quadratic cost accounting. It's more infrastructure for marginal hackathon benefit.
- ISP requires a single additional prompt field (`predicted_group_answer`) and a simple arithmetic aggregation. Much higher innovation/effort ratio.
- That said, QV is listed as a **Week 3 extension** — it's elegant, judge-friendly, and worth adding if time permits.

What about liquid democracy?

- Fractional delegation (Report B) is theoretically powerful but adds significant complexity: you need delegation routing, cycle detection, weight propagation. For a hackathon with 3-5 agents, the agents are already diverse enough that delegation has minimal marginal value.
- **Week 3 extension**: if you add >5 agents, liquid democracy becomes more valuable.

Build effort: 2 days for core ISP + rescaling. +1 day for reputation weighting if Agent Registry is integrated.

Part 5: Expanded Mechanism Repertoire

Report A uniquely proposes mechanisms beyond debate and vote. Report B stays within the debate/vote binary (with innovations within each). The expanded repertoire is a significant differentiator for Innovation scoring.

5.1 Mechanisms to Include

Mechanism	When the Selector Should Route Here	Implementation Complexity	Priority
Structured Debate (factional, adaptive)	High complexity, requires adversarial reasoning, high stakes	Medium (3-4 days)	**Core**
ISP-Weighted Vote	Simpler aggregation tasks, diversity of views matters, time-sensitive	Low (2 days)	**Core**
Delphi Consensus	Subjective/values-laden questions, risk of groupthink in debate	Low-Medium (2 days)	**Extension**
MoA Layered Synthesis	Open-ended generation where synthesis > argumentation	Low (1-2 days)	**Extension**
Quadratic Vote	Multi-option decisions where preference intensity matters	Medium (2-3 days)	**Extension**

5.2 Delphi Protocol (from Report A)

Multiple 2025 papers validate LLM-based Delphi. The flow:

1. Agents independently generate responses (no exposure to others)
2. Responses are anonymized and redistributed as "peer feedback"
3. Agents revise their answers considering anonymous feedback
4. Repeat for 2-3 rounds
5. Final aggregation

Key advantage over debate: Anonymization prevents conformity bias — the core weakness Li et al. identify. Particularly valuable for subjective tasks where debate risks groupthink.

5.3 Mixture-of-Agents Layered Synthesis (from Report A)

The MoA paper (Wang et al., ICLR 2025) achieves 65.1% on AlpacaEval 2.0 vs GPT-4o's 57.5% using only open-source models. Architecture:

- Layer 0: N agents generate independently
- Layer 1: N agents receive ALL Layer 0 outputs and generate refined responses
- Layer 2: Aggregator agent synthesizes Layer 1 outputs into final answer

Key advantage: Iterative refinement through exposure to alternatives, without adversarial dynamics. Best for creative/generative tasks where there's no single "correct" answer.

Part 6: On-Chain Architecture — Comparative Analysis & Recommendation

6.1 Report A's On-Chain Design

- **Merkle tree verification**: Each argument hashed as leaf → tree built per debate → root committed to Solana. Full transcripts stored off-chain (IPFS/Cloud Storage), only hash on-chain.
- **KAMIYO Protocol integration**: Escrow with configurable time-locks, Switchboard oracle quality scoring, PDA agent identities with SOL collateral, ZK-private voting for disputes.
- **x402-solana**: HTTP 402 Payment Required standard for agent-to-agent micropayments. Agents charge per-contribution.
- **SAID Protocol + Agent Registry**: Verifiable on-chain agent identity with reputation.

6.2 Report B's On-Chain Design

- **Solana Agent Registry (ERC-8004 standard)**: Identity Registry (PDA per agent) → Validation Registry (TEE oracle verification) → Reputation Registry (automatic scoring/slashing).
- **Detailed Solana Skills integration**: Security Checklist, Confidential Transfers (Token-2022), Kit↔web3.js interop, IDL & Client Code Gen, Testing Strategy (LiteSVM, Mollusk, Surfpool).
- **Light Protocol ZK Compression**: Compress large transcript hashes into rent-free PDAs.
- **Community skills**: Jupiter, Raydium, Orca for DeFi escrow yield; VulnHunter/Code Recon for security auditing; Pyth/Switchboard for oracle data.

6.3 Assessment

Report B has significantly more granular Solana tooling knowledge — the specific skill names, the 3-layer Helius prompt architecture, the testing strategy breakdown. However, much of it is DeFi-oriented tooling (Jupiter swaps, Raydium CLMM, Orca Whirlpools, ClawPump token launches) that's **irrelevant to your use case**. You're building an arbitration primitive, not a DeFi protocol.

Report A's KAMIYO discovery is the most directly relevant on-chain tool — it already implements escrow + oracle-verified quality scoring + agent identity, which is essentially your smart contract minus the mechanism selector.

6.4 Synthesized On-Chain Recommendation

Core Anchor Contract (custom-built):

```
// Simplified account structure
pub struct TaskAccount {
    pub task_id: [u8; 32],
    pub mechanism: MechanismType,  // Debate | Vote | Delphi | MoA
    pub selector_reasoning_hash: [u8; 32],  // Hash of LLM reasoning for mechanism selection
    pub transcript_merkle_root: [u8; 32],   // Merkle root of all argument/vote hashes
    pub decision_hash: [u8; 32],            // Hash of final answer + metadata
    pub quorum_reached: bool,
    pub agent_count: u8,
    pub consensus_threshold: u8,            // e.g., 60 = 60%
    pub payment_amount: u64,
    pub payer: Pubkey,
    pub mechanism_switches: u8,             // Number of mid-execution switches (novelty!)
    pub created_at: i64,
    pub completed_at: Option<i64>,
}

pub enum MechanismType {
    Debate,
    Vote,
    Delphi,
    MoA,
    Hybrid { primary: Box<MechanismType>, switched_to: Box<MechanismType> },
}

// Instructions:
// 1. initialize_task → creates TaskAccount, locks escrow
// 2. record_mechanism_selection → stores selector reasoning hash
// 3. submit_receipt → stores merkle root + decision hash after completion
// 4. record_mechanism_switch → logs mid-execution mechanism changes (unique feature!)
// 5. release_payment → checks quorum, releases escrow
// 6. get_task_status → view function
```

The `record_mechanism_switch` instruction is novel — no existing system tracks on-chain when and why an AI orchestrator changed its deliberation strategy mid-task. This is a strong Innovation signal.

Tooling integration (prioritized):

Tool	Purpose	Priority	Effort
Anchor (custom contract)	Core escrow + receipt storage + mechanism tracking	Core, Week 1	3-4 days
Helius RPC + Webhooks	Real-time confirmation feeds, WebSocket streaming for dashboard	Core, Week 1-2	1 day setup
Helius Skills (Build skill)	DAS access, WebSocket/Webhook patterns, priority fee optimization	Core, integrated throughout	Install day 1
Solana Agent Kit V2	Pre-built on-chain actions, LangGraph integration template	Core, Week 1	1 day setup
SAID Protocol / Agent Registry	On-chain agent identity + reputation for vote weighting	Extension, Week 3	2 days
x402-solana	Agent-to-agent micropayments per contribution	Extension, Week 2-3	1-2 days
Light Protocol ZK Compression	Compress receipt PDAs to minimize rent costs	Extension, Week 3	1 day
Security Checklist skill	Pre-mainnet audit of Anchor contract	Required, Week 4	0.5 days

What to skip from Report B:

- Jupiter/Raydium/Orca DeFi skills — irrelevant to your use case
- ClawPump token launches — completely irrelevant
- DFlow trading — irrelevant
- Confidential Transfers — nice-to-have but adds complexity for marginal privacy benefit in a demo
- Lulo yield aggregator — you're not optimizing escrow yield during a 5-week hackathon

Helius 3-Layer Prompt Architecture (adopted from Report B): This is a genuinely good idea for ensuring agents interact with Solana correctly:

- **Layer A (Harness):** Runtime-specific tool execution rules in system prompt
- **Layer B (Skills):** Canonical Helius SDK patterns from SKILL.md + references
- **Layer C (Task):** Dynamic context from the mechanism selector about the current task

Install via: `npx skills add helius-labs/core-ai --skill build`

Part 7: Full System Architecture

```
┌──────────────────────────────────────────────────────────┐
│                    TASK INPUT (from user or SDK)          │
└──────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────┐
│              META-ORCHESTRATOR (LangGraph root graph)     │
│                                                           │
│  ┌──────────────────────────────────────────────────┐    │
│  │ Thompson Sampling Bandit + LLM Reasoning Agent   │    │
│  │ Input: task features + historical mechanism performance │
│  │ Output: mechanism + confidence + chain-of-thought │    │
```

```
  → CoT hashed and committed on-chain

        ▼                   ▼                   ▼

  DEBATE        VOTE          DELPHI /
  ENGINE        ENGINE        MoA
 (factional   (ISP-weighted  (extension)
  adaptive)    calibrated)

        ▼                   ▼                   ▼

  MID-EXECUTION STATE MONITOR (runs every round/phase)
  Tracks: disagreement_entropy, information_gain_delta
  Can trigger: mechanism switch, early termination
  Switches logged on-chain via record_mechanism_switch

                            ▼

  TRANSCRIPT HASHER + MERKLE BUILDER
  - Each agent output → SHA-256 leaf
  - All leaves → Merkle tree → root
  - Full transcript → off-chain storage (GCS bucket)

                            ▼

                    SOLANA SETTLEMENT LAYER

 Anchor Contract     Helius Webhooks     Agent Registry
 - TaskAccount PDA   - Confirmation      - PDA identity
 - Escrow              streaming         - Reputation
 - Receipt storage   - Event triggers        scoring
 - Mechanism switch                        (extension)
   tracking

                            ▼

                    FASTAPI BRIDGE (Josh leads)

POST /tasks/            → create task, run mechanism selector
POST /tasks/{id}/run    → execute selected mechanism
GET  /tasks/{id}        → status, transcript, on-chain hash
GET  /tasks/{id}/stream → SSE stream of live debate/votes
POST /tasks/{id}/pay    → trigger escrow release

Deployed on Google Cloud Run (scale-to-zero)
WebSocket/SSE for real-time debate streaming to dashboard

                            ▼

                    SDK + DASHBOARD LAYER
```

```
│                                                                  │
│  ┌──────────────────────────┐  ┌──────────────────────────────┐ │
│  │ agora-sdk (PyPI)         │  │ React Dashboard (Joshua Ddf) │ │
│  │ - LangGraph integration  │  │ - Task submission UI         │ │
│  │ - CrewAI compatibility   │  │ - Live debate streaming      │ │
│  │ - pip install agora-sdk  │  │ - On-chain receipt viewer    │ │
│  └──────────────────────────┘  │ - Benchmark dashboard        │ │
│                                └──────────────────────────────┘ │
└──────────────────────────────────────────────────────────────────┘
```

Part 8: Technology Stack — Final Decisions

8.1 Frameworks

Component	Choice	Rationale
Agent orchestration	LangGraph	Both reports agree. Graph-based architecture maps to debate cycles (cyclic subgraphs), voting (parallel fan-out via Send API), adaptive termination (conditional edges), state persistence. No viable alternative provides this level of control.
LLM SDK	Hybrid: Vertex AI SDK + Anthropic Python SDK	Gemini remains on Vertex; Claude uses direct Anthropic API. Preserves provider routing flexibility while avoiding Vertex-side Anthropic quota constraints.
LLM backbone (generation)	Gemini 2.0 Flash (Vertex) + optional Claude tier (Anthropic)	Cost-efficient baseline generation and voting, with direct Claude fallback/diversity path when configured.
LLM backbone (reasoning)	Gemini 2.5 Pro (Vertex) or Claude Sonnet 4.6 (Anthropic)	Rebuttals, cross-examination, mechanism selection reasoning. Stronger reasoning capability, worth the cost for critical nodes.
LLM backbone (synthesis)	Gemini 2.5 Pro (Vertex) or Claude Sonnet 4.6 (Anthropic)	Final aggregation and trajectory scoring.
Smart contract	Anchor (Rust)	Both reports agree. Typed accounts, IDL generation, constraint macros. Fork escrow example as base.
API	FastAPI	Standard. Josh leads.
Blockchain infra	Helius RPC + WebSockets + Webhooks	Real-time confirmation feeds. Install Build skill day 1.
Agent toolkit	Solana Agent Kit V2	LangGraph multi-agent template included. Modular plugin architecture.
Dashboard	React + SSE	Joshua Ddf leads. Real-time streaming of debate rounds.
Deployment	Google Cloud Run	Scale-to-zero for cost. Min-instances=1 during demo week for WebSocket persistence.

8.2 Key Libraries & Tools

```
# Python environment
langgraph>=0.3.0          # Core orchestration
google-cloud-aiplatform   # Vertex AI SDK for Gemini access
langchain-google-genai    # Gemini via Vertex AI mode
anthropic                 # Direct Claude API client (AsyncAnthropic)
fastapi                   # API layer
uvicorn                   # ASGI server
```

```
solders                    # Solana Python SDK (preferred over solana-py)
anchorpy                   # Anchor client for Python
httpx                      # Async HTTP for Helius API calls
sse-starlette              # Server-Sent Events for real-time streaming
numpy                      # Thompson Sampling math
hashlib                    # SHA-256 (stdlib)
merkletools                # Merkle tree construction (pip install merkletools)

# Rust/Solana
anchor-lang = "0.30+"      # Anchor framework
solana-program             # Solana runtime
spl-token-2022             # Token extensions (if confidential transfers added)

# JavaScript/TypeScript (for Anchor tests + frontend)
@coral-xyz/anchor          # Anchor TS client
@solana/web3.js            # Solana web3
@helius-labs/sdk            # Helius SDK

# Skills to install day 1
npx skills add helius-labs/core-ai --skill build
npx skills add solana-foundation/security-checklist
npx skills add solana-foundation/kit-web3js-interop
```

Part 9: Revised Sprint Plan

Week 1 (April 6-12): Foundations + Core Engines

Dave — Week 1:

- [ ] **Day 1-2: Paper deep-dive + Thompson Sampling bandit**
  - Read Li et al. in full. Annotate: task feature correlations, benchmark splits, martingale discussion.
  - Implement Thompson Sampling mechanism selector (`selector/bandit.py`):
    - Beta(α, β) posteriors per (task_category, mechanism) pair
    - Task feature extraction: complexity, topic_type, expected_disagreement, time_sensitivity
    - ~50 lines of core logic
  - Implement LLM reasoning wrapper (`selector/reasoning_agent.py`):
    - Prompted with task features + bandit recommendation + historical performance
    - Outputs: {mechanism, confidence, chain_of_thought_reasoning}
    - Reasoning gets SHA-256 hashed
- [ ] **Day 3-4: Factional Debate Engine**
  - Build debate engine (`runtime/debate.py`) as LangGraph StateGraph:
    - Faction assignment node: split agents based on initial answer diversity
    - Opening statements node (parallel fan-out)
    - Cross-examination node (Devil's Advocate agent)
    - Rebuttal node (parallel fan-out) with MAD-Oracle state locking for verified claims
    - Convergence check node (conditional edge):
      - Compute disagreement_entropy = -Σ p(answer) * log(p(answer))
      - Compute info_gain_delta = |entropy_current - entropy_previous|
      - If delta < 0.05 for 2 rounds → terminate
    - Final aggregation node with trajectory-aware scoring
- [ ] **Day 5-6: Transcript Hasher + Merkle Builder**
  - Build `runtime/hasher.py` :
    - Deterministic JSON serialization of each agent output

        - SHA-256 hash per output (Merkle leaf)
        - Merkle tree construction → root computation
        - Output: {merkle_root, leaf_count, round_count, quorum_reached, final_answer}
    - Build mid-execution state monitor (`runtime/monitor.py`):
        - Entropy tracking per round
        - Mechanism switch trigger logic
        - Switch logging format for on-chain recording
- [ ] **Day 6-7: ISP Vote Engine**
    - Build vote engine (`runtime/vote.py`) as LangGraph StateGraph:
        - Parallel generation node (each agent outputs answer + confidence + predicted_group_answer)
        - Confidence rescaling node (temperature scaling)
        - ISP aggregation node
        - Quorum check node
        - Merkle commit node

Josh — Week 1:

- [ ] **Day 1-2: Environment setup + Anchor scaffold**
    - All Solana/Anchor tooling installed
    - Fork Anchor escrow example
    - Install Helius Build skill + Solana Agent Kit V2
    - Create monorepo structure
- [ ] **Day 3-5: Anchor smart contract**
    - Implement TaskAccount with all fields (including mechanism_switches counter)
    - Instructions: initialize_task, record_mechanism_selection, submit_receipt, record_mechanism_switch, release_payment, get_task_status
    - Anchor tests in TypeScript
    - Deploy to devnet
- [ ] **Day 6-7: FastAPI scaffold + basic endpoints**
    - POST /tasks/, GET /tasks/{id}
    - Dockerfile + docker-compose
    - GitHub Actions CI

Joshua Ddf — Week 1:

- [ ] React project scaffold
- [ ] Task submission page (basic)

Week 2 (April 13-19): Integration + SDK

Dave — Week 2:

- [ ] **Wire orchestrator end-to-end** (`runtime/orchestrator.py`):
    - Task in → selector decides → engine executes → receipt generated → on-chain commit
    - Mid-execution monitor integrated
    - Test with 20+ synthetic tasks, record accuracy per mechanism
- [ ] **SDK v1** (`sdk/`):
    - `AgoraArbitrator` class with LangGraph native integration
    - `AgoraNode` for drop-in LangGraph graph addition
    - Publish to PyPI as `agora-sdk` 0.1.0-alpha
- [ ] **Benchmark suite**: debate vs. vote vs. ISP-vote vs. selector-chosen on Li et al. benchmark subsets
    - Record: accuracy, latency, token cost, mechanism switch frequency

Josh — Week 2:

- [ ] Wire FastAPI ↔ Solana (submit_receipt, record_mechanism_switch on-chain)
- [ ] SSE streaming endpoint for real-time debate events
- [ ] End-to-end test: task → selection → execution → on-chain receipt → Solana explorer verification
- [ ] Cloud Run deployment

Both — Week 2:

- [ ] SWARM Discord traction push: offer SDK integration to 2-3 teams
- [ ] x402-solana exploration for agent-to-agent payments

Week 3 (April 20-26): Polish + Extensions + Traction

Dave — Week 3:

- [ ] SDK polish: CrewAI compatibility, OpenAI Agents SDK compatibility, README + examples
- [ ] **Extension: Delphi mechanism** (if ahead of schedule — 2 days)
- [ ] **Extension: Quadratic voting variant** (if ahead of schedule — 1-2 days)
- [ ] **Extension: Agent Registry reputation weighting** for vote engine (1-2 days)
- [ ] 2-page technical writeup: architecture, mechanism selector accuracy, debate vs. vote performance, on-chain overhead analysis

Josh — Week 3:

- [ ] Reliability hardening: retry logic on Solana txns, circuit breaker on agent calls, task timeout
- [ ] Light Protocol ZK Compression for receipt PDAs (if time permits)
- [ ] Devnet → mainnet prep: contract audit using Security Checklist skill
- [ ] Structured logging + Cloud Monitoring

Joshua Ddf — Week 3:

- [ ] Full dashboard: live debate streaming, on-chain receipt viewer, benchmark dashboard
- [ ] Dark theme, real-time WebSocket feel

Week 4+ (April 27 – May 11): Traction + Submission

- [ ] Mainnet deployment with tiny real payments (0.01 SOL)
- [ ] Active traction push: target 3-5 SDK users, 50+ agent-to-agent transactions
- [ ] Demo video: full flow < 3 minutes
- [ ] Pitch video: 2-3 minutes covering research insight → implementation → product wedge → traction
- [ ] GitHub repo cleanup: public, clean README, working instructions
- [ ] SWARM CLI: submit updates regularly (hidden prizes for activity)

Part 10: Innovation Narrative for Judges

The system's innovation story has three pillars that map directly to the 40% Innovation criterion:

Pillar 1: "The Orchestrator That Reasons and Learns" Unlike any existing multi-agent framework, the mechanism selector is itself an intelligent agent that provides chain-of-thought justification for *why* it chose debate over voting (or vice versa), learns from outcomes via Thompson Sampling, and can reverse its decision mid-execution if convergence metrics indicate the wrong mechanism was chosen. Every piece of this meta-reasoning is cryptographically committed on-chain. No one has built a self-improving, explainable, on-chain mechanism selector before.

Pillar 2: "Debate That Actually Works" We directly address the documented failure of naive multi-agent debate (the martingale property from Li et al. 2025) with three structural interventions: factional assignment that makes sycophantic convergence architecturally impossible, Devil's Advocate cross-examination that forces genuine adversarial engagement, and information-theoretic adaptive termination that stops wasting tokens when debate has plateaued. This isn't prompting tricks — it's structural redesign backed by FREE-MAD, MACI, and ACC-Debate research.

Pillar 3: "Proof-of-Deliberation as an On-Chain Primitive" Every step of the AI governance process — from mechanism selection rationale to individual arguments to vote tallies to mechanism switches — is Merkle-hashed and committed to Solana. This creates a new primitive: **verifiable AI collective decision-making**. Any third party can recompute the hashes from the off-chain transcript and verify against the on-chain Merkle root. The `record_mechanism_switch` instruction is entirely novel — tracking on-chain when and why an AI orchestrator changed its deliberation strategy.

Appendix A: Key Papers Referenced

Paper	Year	Key Insight for This System
Li et al., "Debate or Vote?" (NeurIPS 2025 Spotlight)	2025	Debate induces martingale; majority voting accounts for most gains
FREE-MAD (Cui et al.)	2025	Anti-conformity + trajectory scoring: +16pp accuracy, -50% tokens
MACI (Chang, Stanford)	2025	Dual-dial adaptive termination via information gain plateau detection
Tree-of-Debate (Kargupta et al.)	2025	Hierarchical argument trees replace flat round-robin
ACC-Debate (Estornell et al., ICLR 2025)	2025	Actor-critic training of debate as learned skill
ReConcile (Chen et al., ACL 2024)	2024	Confidence rescaling + architectural diversity: +7.7% over debate
ISP / Optimal Weight (Pan et al.)	2025	Second-order information (predicted group answer) provably beats majority vote
MoA (Wang et al., ICLR 2025)	2025	Layered synthesis: 65.1% AlpacaEval with open-source models
Bayesian Orchestration of Multi-LLM Agents	2026	34% cost reduction via Bayesian belief aggregation
BaRP (Bandit-based Routing)	2025	12.46%+ improvement over offline classifiers under partial feedback
CoCoMaMa (HyperAgents 2025)	2025	Volatile bandit arms — add new mechanisms at runtime without retraining
TED Framework	2025	Factional debate + GAT adjudication (adopted structurally, simplified implementation)
STRMAC	2025	Continuous state-aware routing via cosine similarity (concept adopted, implementation simplified)

Appendix B: Links to Solana/Helius Agent Tooling

Resource	URL	Use
Solana Agent Skills	solana.com/skills	60+ community-contributed drop-in agent modules

Resource	URL	Use
Helius Agent Skills	helius.dev/docs/agents/skills	3-layer prompt architecture, Build/Phantom/DFlow/SVM skills
Solana Agent Kit V2	github.com/sendaifun/solana-agent-kit	60+ on-chain actions, LangGraph multi-agent template
KAMIYO Protocol	kamiyo.ai	Escrow + oracle quality scoring + ZK voting + reputation
x402-solana	github.com/payainetwork/x402-solana	HTTP 402 agent-to-agent micropayments
SAID Protocol	saidprotocol.com	On-chain agent identity infrastructure
Solana Agent Registry	solana.com/agent-registry	Verifiable identity + portable reputation
Light Protocol	lightprotocol.com	ZK Compression for rent-free PDAs
Helius MCP Server	helius.dev/agents	Claude Code plugin, MCP tools for live blockchain data
Awesome Solana AI	github.com/solana-foundation/awesome-solana-ai	Curated list of Solana AI tooling

---

Last updated: April 10, 2026 Synthesized from: Report A (Claude Research) + Report B (Parallel Agent Research) Team: Dave · Josh · Joshua Ddf